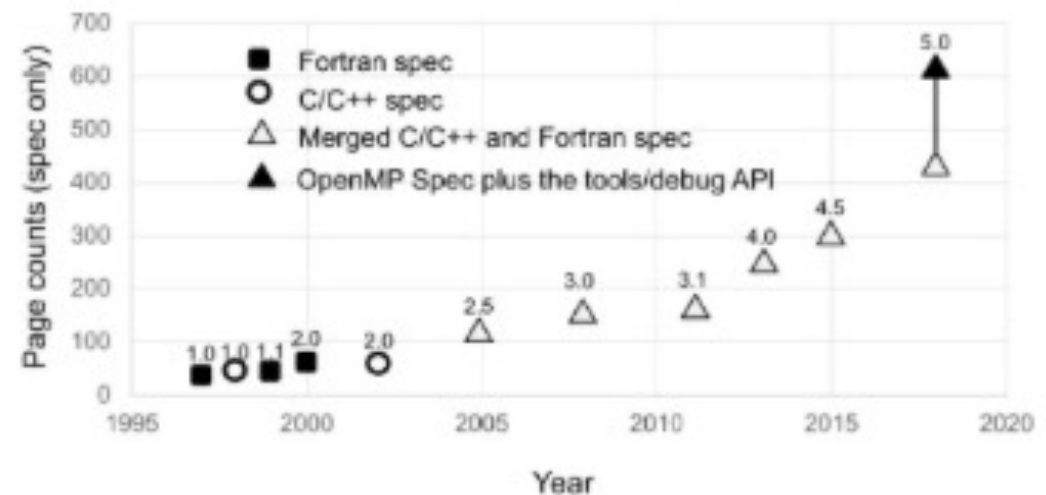




OpenMP

the common core

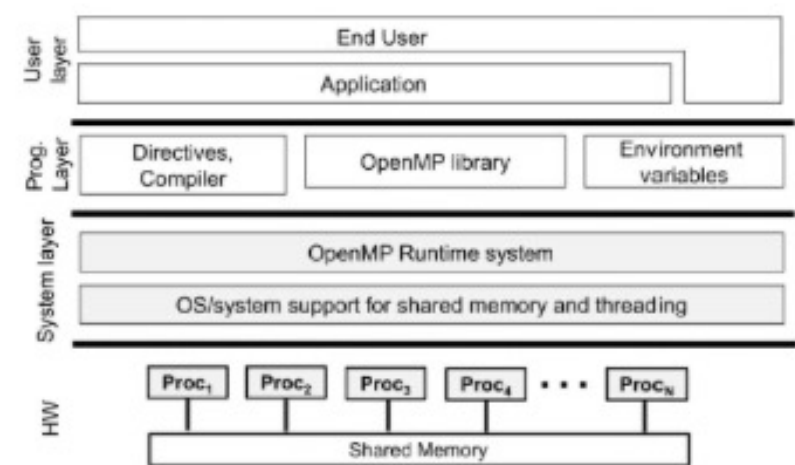


OpenMP Standard (version 2.5)

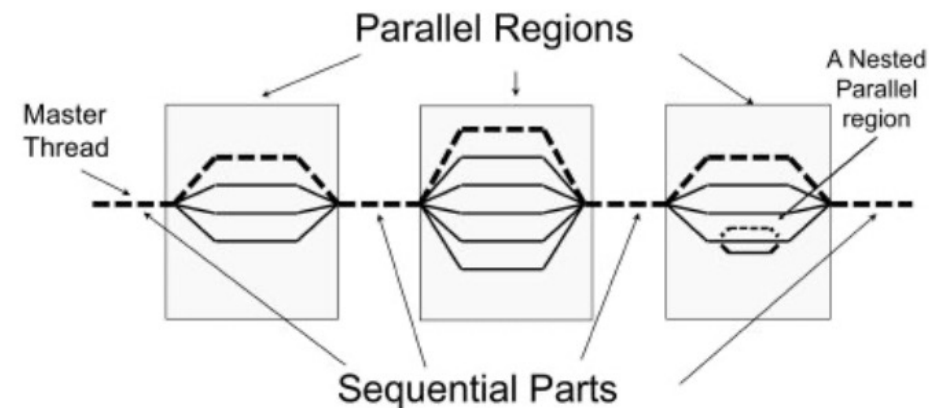
Introduction to OpenMP

- OpenMP is a standard to **Shared Memory (SM) parallel programming** (e.g., targets multi-core machines).

- Based on: Compiler directives, Library routines and Environment variables;
- Supports C/C++ and Fortran programming languages.



- Execution model is based on the **fork-join** model of parallel execution.



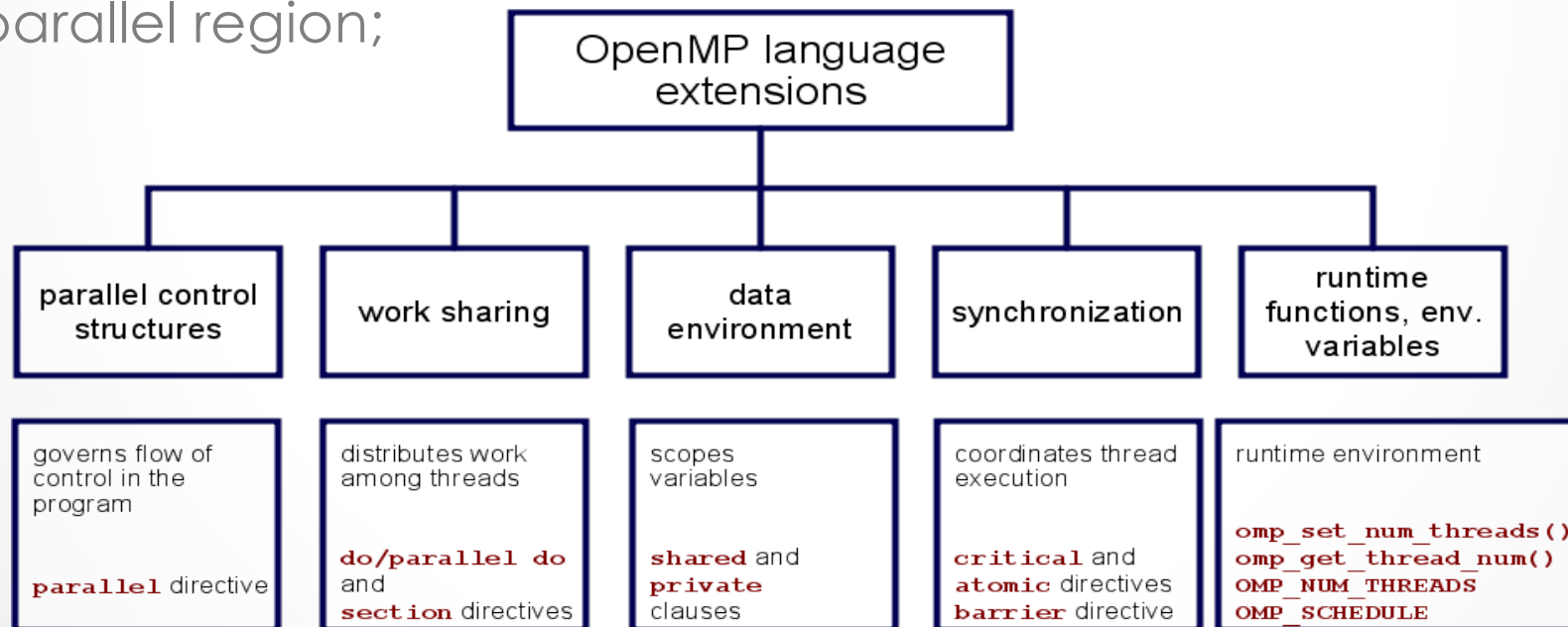
- Parallelism is specified through directives, added by the programmer to the code
 - The compiler implements the parallelism (e.g, POSIX Threads)

OpenMP considerations:

- It is the programmer's responsibility to ensure **correctness** and **efficiency** of parallel programs
 - OpenMP itself **does not** solve problems as :
 - **Data races**, deadlock or poor load balancing (among others).
 - But, offers routines to solve problems like load balancing.
- The creation/managing of threads is delegated to the compiler & OpenMP runtime:
 - + Easier to parallelize applications;
 - - Less control over threads behaviour.
- By default, the number of **parallel activities** is defined according to available resources
 - e.g. 2 cores -> 2 threads
 - SMT capability counts as additional cores

OpenMP: Programming Model

- An OpenMP program begins with a single thread (**master thread**).
- **Parallel regions** create a team of parallel activities;
- **Work-sharing** constructs generate work for the team to process;
- **Data sharing** clauses specify how variables are shared within a parallel region;



The OpenMP core constructs (1)

- OpenMP directives format for C/C++ applications:
 - `#pragma omp` directive-name [clause[[,] clause]...] new-line
- Parallel Construct => required on every OpenMP program
 - `#pragma omp parallel` Creates a team of threads.
- Work-sharing Constructs
 - `#pragma omp for` Assignment of loop iterations to threads.
 - ~~◦ `#pragma omp sections` Assignment of blocks of code (section) to threads.~~
 - `#pragma omp single` Restricts a block of code to be executed by a single thread.

The OpenMP core constructs (2)

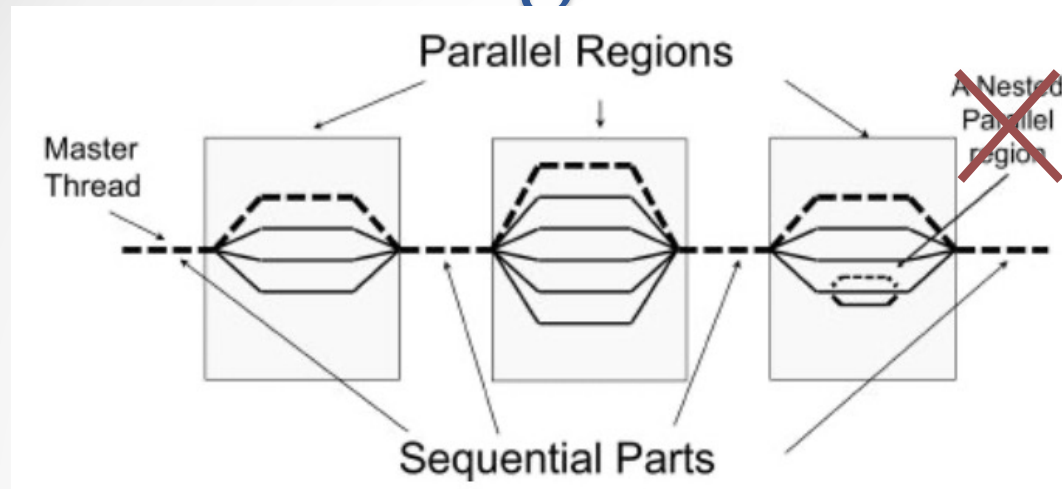
- Tasking Constructs (standard v2.5)

- `#pragma omp task` Creation of a task to be executed by threads in the team.

- Master & Synchronization Constructs

- ~~`#pragma omp master` Restricts a block of code to be executed only the master thread.~~
- `#pragma omp critical` Restricts the execution of a block of code to a single thread at a time.
- `#pragma omp barrier` Makes all threads in a team to wait for the remaining.
- `#pragma omp taskwait` wait for the completion of the current task child's.
- `#pragma omp atomic` Ensures that a specific storage location is managed atomically.
- ~~`#pragma omp flush` Makes a thread's temporary view of memory consistent with memory.~~
- ~~`#pragma omp ordered` Specifies a block of code in a loop region that will be executed in the order of the loop iterations.~~

Parallel Region



- When a thread encounters a parallel construct, a **team of threads** is created (**FORK**);
- The thread which encounters the parallel region becomes the **master** of the new team;
- **All threads** in the team (including the master) execute the region;
- At end of parallel region, all threads synchronise, and join master thread (**JOIN**).

Parallel region syntax

```
#pragma omp parallel [clauses]  
{  
    code_block  
}
```

Where clause can be:

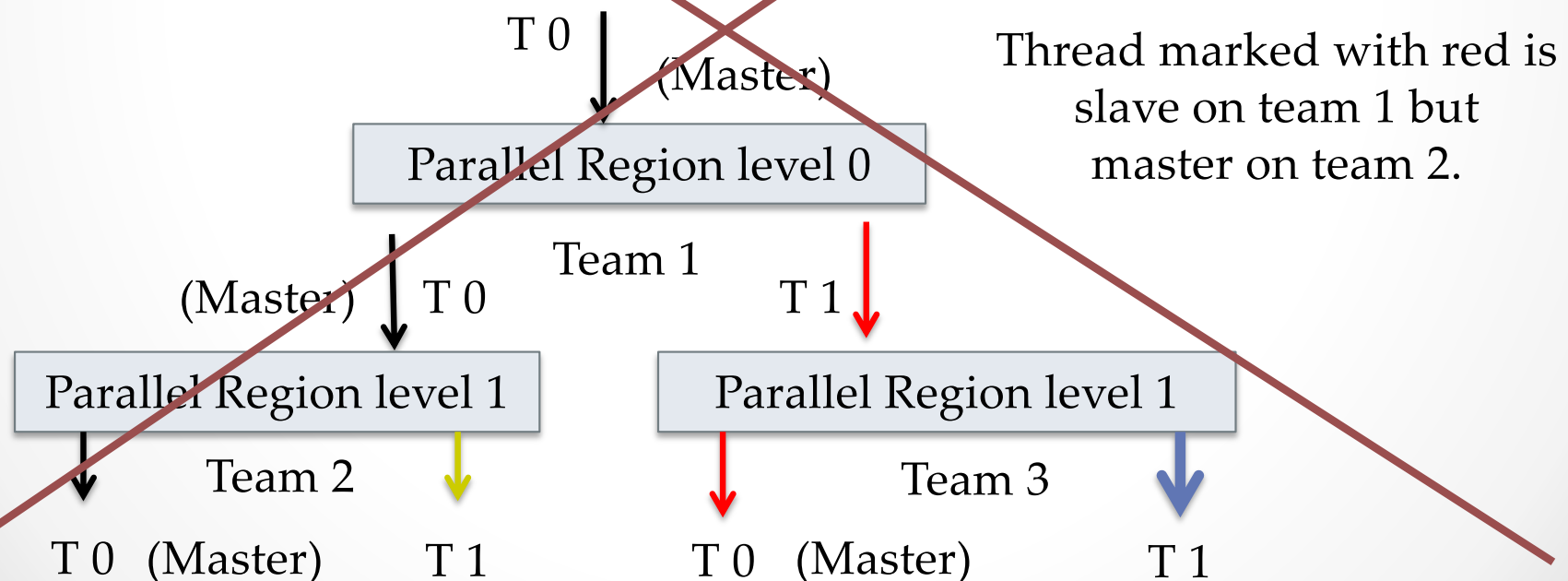
```
if (scalar-expression)  
num_threads (integer-expression)  
private (list)  
firstprivate (list)  
shared (list)  
reduction (operator: list)
```

Data clauses (next week)

~~Nested Parallel Region => (outdated)~~

nested parallelism => use tasks instead (next lecture)

- If a thread in a team executing a parallel region encounters another parallel directive, it creates a new team, and becomes the master of this team;
- If **nested parallelism** is disabled, then no additional team of threads will be created.
- To enable/disable -> `omp_set_nested(x);`
- **Example:**



Work-Sharing: Loop Construct

Data-parallelism!

- The *for* loop iterations are distributed across **threads in the team**;
 - The distribution is based on:
 - **chunk_size**, by default is 1;
 - **schedule** by default is **static**.
- Loop **schedule** (map of loop iterations to threads):
 - **Static** – Iterations divided into chunks of size **chunk_size** assigned to the threads in a team in a **round-robin** fashion;
 - **Dynamic** – the chunks are assigned to threads in the team as the threads request them;
 - **Guided** - similar to dynamic but the chunk size decreases during execution.
 - **Auto** – the selection of the scheduling strategy is delegated to the OpenMP implementation.

Loop work-sharing syntax

```
#pragma omp for[clauses]
{
    code_block
}
```

Where clause can be:

```
private(list)
firstprivate(list)
lastprivate(list)
reduction(operator: list)
schedule(kind[, chunk_size])
collapse(n)
ordered
nowait
```

← no implicit barrier

There is an implicit *barrier* and the end of the *for*

Loop Constructors

- schedule(**static**) vs schedule(**dynamic**)
 - **Static** has lower **overhead**;
 - **Dynamic** has a better load balance approach;
 - Increasing the chunk size in the dynamic for:
 - Diminishing of the scheduling overhead;
 - Increasing the possibility of load balancing problems.
- Lets consider the following loop that we want to parallelize using 2 threads, being `void f(int i)` a given function

```
#pragma omp parallel for schedule ( ? )  
for(i = 0; i < 100; i++)  
    f(i);
```

What is the most appropriated type of scheduling?

Parallel for with ordered clause

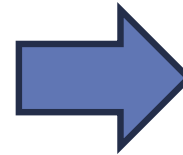
- `#pragma omp for schedule(static) ordered`
for (i = 0; i < N; ++i)
{
 ... // do something here (in parallel)
 `#pragma omp ordered`
 {
 printf("test() iteration %d\n", i);
 }
}

~~Parallel execution of code sections~~-use the task model instead (next lecture)

- ~~Supports heterogeneous tasks:~~ • Implementation with tasks:

```
#pragma omp parallel  
{  
#pragma omp sections  
{  
#pragma omp section  
{  
taskA();  
}  
#pragma omp section  
{  
taskB();  
}  
#pragma omp section  
{  
taskC();  
}  
}
```

Function-parallelism!



```
#pragma omp parallel  
{  
#pragma omp single  
{  
#pragma omp task  
{  
taskA();  
}  
#pragma omp task  
{  
taskB();  
}  
#pragma omp task  
{  
taskC();  
}  
}  
}
```